

CaseWhy — Pre-Answered Q&A for the USCIS Torch Demo

1. Every answer below is drawn from `docs/uscis/technical-brief.md` (which has the file/function citations) and was verified against the live application, not recalled from memory.
2. Format: plain-language answer first, then the technical specifics, then where it's shown live in the demo.
3. USCIS evaluates five pass/fail criteria for the Case Status API demo (confirmed directly against `developer.uscis.gov`, not assumed):
 1. Data entry usability.
 2. JSON payload conversion.
 3. OAuth 2.0 authentication.
 4. HTTPS response handling (success and error, message shown in the UI).
 5. Case status tracking.
4. File upload is a separate FOIA API criterion and is not part of this demo.
5. **Round 130** — the actual USCIS API Demo Checklist, sent with the scheduling link, is broader than those five numbered criteria and adds real, separate asks this doc now also answers:
 1. The browser's own DevTools Network tab specifically (see the Network-tab question under "JSON payload handling").
 2. CaseWhy's own frontend/backend authentication as its own point, distinct from USCIS OAuth (see "App authentication").
 3. Manually running the scheduled batch job live (see "Batch/scheduled updates").

About the demo itself

What's the format?

1. A 30-minute Microsoft Teams call, recorded, audio required.
2. Peter (founder, sole operator, and the affidavit's signatory) attends alone.

3. USCIS explicitly allows the same person to be both the primary business contact and the technical demo lead.

What's the `demo_id`, and why does it matter before the call is even scheduled?

1. USCIS assigns a 4-digit ID that must be sent as a `demo_id` header on every Case Status API request during the demo window.
2. The scheduler link isn't issued until we send proof (a screenshot or code snippet) that it's wired in.
3. `src/lib/uscis/client.ts`'s `demoIdHeader()` sends it automatically on every call whenever `USCIS_DEMO_ID` is set.
4. It sends nothing extra when `USCIS_DEMO_ID` isn't set — so it's genuinely off outside the demo window, not just unused.

What happens after the call?

1. Results by email within 2 business days.
2. A fail routes to a 15-minute "Re-Demo Office Hours" covering only the missed criteria — not a full re-demo from scratch.

Architecture and hosting

What is CaseWhy, and who built and operates it?

1. CaseWhy is a free web app that tracks a USCIS case by receipt number and explains its status in plain language.
2. It's built and operated by one person — Peter, founder of CaseWhy LLC, a Florida company.
3. Development is AI-assisted (Claude Code).
4. *Shown in demo: introduction.*

What's the stack, and where does the API call actually happen?

1. Next.js on Vercel, Postgres on Neon, Postmark for email, Stripe for billing.
2. Every call to USCIS's API happens entirely server-side — the browser never sees a credential or a token.
3. Technically: `src/lib/uscis/client.ts` runs only on Vercel's Node.js runtime (never the browser, never an Edge function), so USCIS credentials

never leave our server.

4. *Shown in demo: step 2 (a live status check, watched in server logs).*

Is there a mobile app?

1. No native app.
2. CaseWhy is installable as a Progressive Web App (add to home screen, works offline for cached content), same codebase as the website.

App authentication (frontend/backend) — separate from USCIS OAuth

How does a user's browser authenticate to your own server? Round 130.

1. Neon Auth, supporting password and magic-link sign-in.
2. It's a real session between the browser and our server, checked on every request that touches account or case data (tracking a case, checking status on demand, account settings).
3. This is a completely separate relationship from the one below: it's the browser talking to *us*.
4. The OAuth 2.0 flow in the next section is *us* talking to *USCIS* — two different authentications, not one.
5. *Shown in demo: step 1.*

Why does that distinction matter for this demo?

1. Because the checklist asks for both, as separate lines.
2. "Authentication between the frontend and backend of your application" (this) and "OAuth 2.0" (next section) are different requirements, not two ways of describing the same thing.

OAuth 2.0

Which grant type do you use, and how are tokens handled?

1. Client Credentials, exactly as USCIS documents it.

2. We cache the access token in memory and refresh it automatically about 30 seconds before it expires, rather than fetching a new one on every call — so we're not hammering the token endpoint.
3. Sandbox and production use entirely separate credential pairs, selected by one environment setting; there's no code path where they could get mixed up.
4. *Shown in demo: step 2, watching the token fetch/cache behavior in logs.*

What happens on a 401 or an expired token mid-request?

1. The next call simply re-authenticates.
2. Our cache check runs on every call, so an expired token is caught and refreshed automatically, not retried in a loop.

Where are the client ID and secret stored?

1. As encrypted environment variables in Vercel, never in our source code or version control.
2. `.gitignore` excludes every environment file, and there are zero hardcoded secrets anywhere in the codebase.

JSON payload handling

Which fields do you actually use, and how do you handle the response?

1. Form type, current status text and description, submitted/modified dates, and the full status history array.
2. We parse the response against a defined shape that accounts for USCIS's documented variations (like the IOE-prefix response sometimes omitting dates) rather than assuming every field is always present.
3. *Shown in demo: step 3, a tracked case's status/history display.*

Do you show USCIS's status text as-is, or rewrite it?

1. We show it exactly as USCIS returns it.
2. Separately — visually distinguished — we show our own plain-language explanation of what it means.

3. We never present a rewritten version of USCIS's own words as if USCIS said them.

Do you store the raw API response?

1. We store the fields we use (status text, history entries, dates) — not an opaque full-payload blob.
2. The receipt number itself is stored encrypted, separately from status data.
3. The receipt number is never included in what we send to our AI explanation service (more in the privacy section below).

Can you actually show the JSON payload being sent, not just describe it?

1. Yes — the on-demand check button has a "Show technical details" toggle that renders the exact outbound request (method, URL, headers, with the token redacted) our code sends for that click.
2. It's generated by the same function `getCaseStatus()` itself uses, not a second hand-written description that could drift out of sync.
3. *Shown in demo: step 2.*

Can you show this in the browser's own DevTools Network tab, since that's specifically requested? Round 130.

1. Honestly: the request to USCIS itself, no.
 2. Our OAuth credentials and the USCIS API call run entirely server-side, by design, specifically so a client ID, secret, or access token can never reach the browser.
 3. That means the raw USCIS request genuinely never crosses the browser's own network layer, so DevTools can't show it no matter how the app is built.
 4. What DevTools *does* show, and what I'll point to live: the request from the browser to our own server that triggers that server-side call — the browser-to-us leg is real and visible.
 5. The us-to-USCIS leg is shown instead via the "Show technical details" panel above, built from the same code that makes the real call.
 6. *Shown in demo: step 2.*
-

Error handling

What happens with an invalid or unknown receipt number?

1. The format (3 letters + 10 digits) gets real-time client-side feedback as you type — a visual hint, not a hard block, since a well-formed number still might not be real.
2. We confirmed the real API responses directly against your sandbox.
3. A malformed number returns a real `422` with a message explaining the format requirement.
4. A well-formed but unknown number returns a real `404` with a message saying the case isn't recognized.
5. Both get surfaced to the user as a clear, USCIS-worded error message.
6. *Shown in demo: step 4, deliberately entering a malformed number and an unknown one.*

Does the error message actually reach the screen, or just the logs?

1. It reaches the screen.
2. The on-demand check button shows USCIS's actual message directly, not a generic substitute.

What about rate-limit (429), server-error (5xx), or timeout responses?

1. Every one of those goes through the exact same error-display path as the 422/404 above — there's no special-casing that could silently diverge.
2. We've directly observed a real 429 once during sandbox testing (see the rate-limiting section) and it was handled cleanly.
3. 5xx and auth-failure responses haven't occurred live but would route through the identical code.
4. A 15-second client-side timeout was added specifically so a hung request can't wait forever.
5. *See the response-code table in technical-brief.md, Section 4.*

Do you retry on failure?

1. No automatic retries in the API client.
2. A failed call is reported as an error for that specific check, so we're not compounding load on USCIS's API with a retry storm.

3. The next scheduled check will naturally try again.

Do receipt numbers ever end up in logs?

1. No.
 2. Our status-check job only holds a decrypted receipt number in memory for the duration of the actual API call.
 3. Error logs reference the internal case record, not the receipt number in plaintext.
-

Rate limiting and volume

What's your current and projected call volume, and is there a real limit, not just a documented plan?

1. Today: a small number of daily calls, one per actively tracked case, on a fixed schedule.
2. Volume scales with how many cases our users track, not with site traffic.
3. There's a real, enforced hard cap in the client itself, not just documented pacing: no more than 1,000 calls per day and no more than 4 per second.
4. That cap is checked before every call and the call is refused (with a logged alert) if either limit would be exceeded.
5. Those thresholds match the sandbox's own confirmed limits; we'll raise them once USCIS shares the real production quota.
6. See *"Rate limiting and volume" in technical-brief.md*.

Have you actually load-tested against the sandbox?

1. Yes — a real 5-business-day sandbox volume test, September 14–18, 2026, using our actual production code path, not a mock.
2. ~900 calls/day, under the sandbox's published 1,000/day, 5-TPS ceiling.
3. One real 429 occurred once, Tuesday, one call out of 50 in a single batch.
4. The next scheduled batch completed cleanly, and it hasn't repeated.
5. We're telling you that rather than claiming a suspiciously perfect record.
6. See *"Sandbox testing" in technical-brief.md for current day-by-day numbers*.

Do you cache results, or hit the API on every page load?

1. Results are cached in our own database between scheduled checks.
 2. A user viewing their dashboard isn't triggering a new USCIS API call; they're seeing the last fetched result.
-

Case status tracking

How does a user add a case, and how many can they track?

1. Sign in, enter a receipt number, done.
2. Free accounts get 3 receipt numbers for the life of the account — tracking one or just looking up its status both count against that same cap, and untracking a case doesn't free a slot back up.
3. Paid accounts: more, in tiers, with a hard ceiling above which we talk to the user directly rather than self-serve.
4. *Shown in demo: step 2 (adding a new case live).*

How do you detect a status change, and how does the user find out?

1. Our scheduled job compares the freshly fetched status text against the last one we saw for that case.
2. On a genuine change (never on the very first check), we send an email via Postmark and, if the user's enabled it, a push notification.
3. *Shown in demo: step 5, and run manually live in step 6.*

What happens when a user removes a case?

1. Immediate and self-serve.
 2. One click on the dashboard deletes that case's record, including its encrypted receipt number, from our database right away.
 3. *Shown in demo: step 5.*
-

Batch/scheduled updates — Round 130

Does your app do batch updates? How often?

1. Yes — once a day, on a fixed schedule, every day.

2. It re-checks every actively tracked case against USCIS and sends the notification described above on a genuine change.

Can you run it live, right now, not just describe it?

1. Yes — it's a real endpoint (`/api/cron/check-status`).
2. It's normally called by an external scheduler on a bearer-token header, the same one an admin diagnostic tool uses to trigger and verify real deployed routes.
3. I can call it by hand with the identical header, live, right now.
4. *Shown in demo: step 6.*

How does it handle one case failing without breaking the whole run?

1. Each tracked case is checked independently, inside its own try/catch.
2. A failure on one case is recorded and the loop continues to the next, not aborted.
3. The response reports exactly how many succeeded, how many failed and why, and how many are left if the run's own time budget ran out before finishing the full list.
4. *Shown in demo: step 6, with a deliberately-unrecognized receipt number tracked specifically to produce a real error in the same run as real successes.*

Is that deliberately-broken test case going to look bad, like something's wrong with the app?

1. No — it's tracked on purpose, precisely because it reliably reproduces a real USCIS 404 on demand, so the error-handling path can be shown live rather than described.
2. It's the same code path a genuinely unrecognized case would hit in production, working exactly as intended: reported cleanly, isolated from the cases that succeed.

File upload (not part of this demo — FOIA API criterion, mentioned for completeness only)

Is the document upload feature real, or a placeholder?

1. It's real — paid-tier users can upload PDFs, JPEGs, or PNGs up to 10MB, stored privately (never a public URL).
 2. The original filename is encrypted before storage, since a filename can itself be identifying.
 3. No uploaded document is ever sent to USCIS or any other third party — it's purely for the user's own reference.
 4. We're not planning to show this live, since it's evaluated under the FOIA API's criteria, not Case Status — happy to describe or demo it separately if asked.
-

UI usability

Is the app accessible?

1. We use semantic HTML and ARIA labeling throughout the interface, and it's fully usable via keyboard.
2. Honestly: we don't yet run automated accessibility testing (like axe or Lighthouse CI) as part of our build process.
3. That's a real, acknowledged gap, not something we're claiming full formal compliance on.

What languages does it support?

English and Spanish, with a switcher available on every public and signed-in page.

How fast can a new user get from landing to a tracked case?

1. Under two minutes.
 2. No account required to browse reference material (processing times, Visa Bulletin, the Get Help directories).
 3. Signing up and adding a first case is a short, linear flow.
 4. *Shown in demo: the full flow, start to finish.*
-

Privacy, security, and data handling

How is a receipt number protected?

1. Encrypted with AES-256-GCM before it's ever written to our database — an authenticated encryption mode, so tampering is detected, not silently accepted.
2. The encryption key lives only as a protected environment variable, separate from the database itself.
3. No CaseWhy staff member can view a receipt number in plaintext through any administrative tool, ever.

What exactly gets sent to your AI provider?

1. Form type, processing center, dates, status text, history, and general policy background — never the raw receipt number.
2. That's not a policy promise; it's how the prompt-building code is actually written.
3. The one exception: our paid-tier letter-drafting tool includes the receipt number in the letter itself, since the letter has to name the case to USCIS or a congressional office — sent to the same AI provider solely to help draft that letter.

What happens if a user wants their data deleted?

1. Removing an individual case is immediate and self-serve.
2. Full account deletion is a request — email privacy@casewhy.com and it's completed within 30 days.

Who else receives any case data?

1. USCIS itself (to retrieve status).
2. Anthropic (status text, dates, and history for AI explanations, plus the receipt number specifically for the letter-drafting exception above).
3. Postmark (the notification email address and status text, to send emails).
4. Vercel and Neon (infrastructure — hosting and database, no independent access to decrypted data outside our own code).
5. Stripe (billing information only, never case data).
6. For a paid-tier congressional-representative lookup only — the U.S. Census Bureau's public geocoding API.
7. No advertising, no data brokers, no sale of personal information, ever.

What's your security posture, honestly?

1. HTTPS/TLS everywhere.
 2. Security headers (HSTS, a Content-Security-Policy, and the standard set) on every response.
 3. Encryption at rest for sensitive fields.
 4. A single fail-closed admin allowlist.
 5. No hardcoded secrets anywhere in the codebase.
 6. Automated dependency monitoring runs via Dependabot, with `npm audit` also visible in CI.
 7. One honest, tracked-not-hidden gap: two pre-existing findings in transitive dev-tooling dependencies (not runtime application code) are open pending a deliberate major-version upgrade we didn't want to force through days before this demo.
-

Business and operations

How does CaseWhy make money, and is anything from the USCIS API paywalled?

1. A free tier and an optional paid tier (\$9.99/month, or discounted 6-month/annual).
2. Nothing from the Case Status API is ever paywalled — status checking is free at every tier.
3. The paid tier adds more tracked cases, unlimited AI questions, on-demand checks, document storage, and drafting tools.

Is billing live, or still in test mode?

1. Test mode — we haven't processed a real production payment yet.
2. That's a deliberate, honest state, not something we're overstating; flipping to live payments is a distinct step we haven't taken.

What's your support setup?

1. 13 real, monitored email addresses on our domain, including dedicated ones for security, privacy, legal, and abuse reports.
2. Each is actively polled, with the most sensitive ones escalated immediately.

What happens if production access were ever revoked?

1. We'd show users an honest degraded-service message.
2. We have no scraping fallback and won't build one — if the API isn't available to us, the feature isn't available, full stop.